

Clone a Git Repository

By: Michael Gomez

Table of contents

About.....	3
Reference.....	4
Cloning a Git repository.....	7

Sample

About

Git is a free and open source distributed version control system (VCS) designed to be used with projects of all sizes with speed and efficiency. When installed on your computer, Git will keep track of changes made to a *repository* that you clone locally to your computer. After changes are recognized, you can send or *push* those changes back to the Git repository.

Sample

Reference

The following table is a glossary of Git terms and their descriptions.

Term	Description
alternate object database	Via the alternates mechanism, a repository can inherit part of its object database from another repository, which is called an "alternate".
bare repository	A bare repository is normally an appropriately named directory with a <code>.git</code> suffix that does not have a locally checked-out copy of any of the files under revision control.
blob	Binary Large Object. An object type used to store the contents of a single file. It does not contain metadata such as file name or permissions.
branch	An active line of development. The most recent commit on a branch is referred to as the tip or head of that branch.
cache	<i>Obsolete.</i> A historical synonym for the index .
checkout	The action of updating all or part of the working tree with a tree object or blob from the object database, and updating HEAD and the index.
cherry-pick	The command or action to extract the change introduced by an existing commit and record it as a new commit on top of the current branch tip.
clean	A working tree is clean if it exactly matches the revision referenced by the current HEAD (no uncommitted or modified tracked files).
clone	A local copy of a repository, complete with all of its history, branches, and objects, usually linked back to the original via a remote named <code>origin</code> .
commit (Noun)	A single point in the Git history; a snapshot representing the entire project history up to that point.

Term	Description
commit (Verb)	The action of storing a new snapshot of the project's state in the Git history by creating a new commit object.
commit graph	A Directed Acyclic Graph (DAG) structure formed by the interrelated commits in the object database.
detached HEAD	A state where HEAD directly references an arbitrary commit rather than pointing to a named branch. Commits made here do not belong to any branch.
dirty	A working tree is considered dirty if it contains modifications that have not yet been committed to the current branch.
fast-forward	A special type of merge where the target branch is a direct descendant of the current branch, allowing Git to simply move the branch pointer forward without creating a new merge commit.
fetch	Fetching a branch means getting the branch's head ref from a remote repository, downloading any missing objects into the local database, and updating remote-tracking refs.
hash	In the context of Git, a synonym for an object name (the 40-character SHA-1 or 64-character SHA-256 string).
head / ref	A named reference to the commit at the tip of a branch.
HEAD	The pointer to the current active branch or commit. Your working tree is normally derived from the state of the tree referred to by HEAD.
hook	A script that Git automatically executes before or after certain key operations (such as committing, pushing, or receiving).
index	A collection of files with stat information, whose contents are stored as objects. It acts as the staging area between your working tree and your next commit.

Term	Description
merge	As a verb: to bring the contents of another branch into the current branch. As a noun: a successful merge that results in a "merge commit" representing the combined states.
object	The fundamental unit of storage in Git. Objects are immutable and uniquely identified by the cryptographic hash of their contents.
object database	Stores the set of objects (commits, trees, blobs, tags) making up the repository history, usually located in <code>.git/objects/</code> .
origin	The default upstream repository. Most projects have at least one upstream project they track; by default, Git names this location <code>origin</code> .
pull	The action of pulling a branch, which means to fetch it and immediately merge it into the current branch.
push	Pushing a branch means uploading local branch refs and missing objects to a remote repository, updating the remote head ref if it is a safe fast-forward.
rebase	To reapply a series of changes from a branch onto a different base tip, effectively rewriting history to create a linear progression.
repository	A collection of refs together with an object database containing all historical objects, along with meta-information.
tag	A static reference pointing to a specific object (usually a commit) to permanently mark a significant point in history (e.g., a release version like <code>v1.0</code>).
working tree	The actual directory of files that you are currently visible seeing and editing on your local machine, usually derived from <code>HEAD</code> .

Cloning a Git repository

Before you begin: You must have installed Git on your computer locally and you have obtained a Git repository URL to clone. You can use Git through a command line interface such as PowerShell or through a graphical user interface such as GitHub Desktop.

About this task: The following steps will demonstrate how to clone a Git repository using GitHub Desktop.

1. In GitHub Desktop, navigate to **File > Clone repository...** or select **Clone a repository from the Internet...**
2. On the **Clone a repository** screen, select the **URL** tab.
3. In the **URL** tab, under **Repository URL or GitHub username and repository**, enter the Git repository URL you have obtained.
For example, a Git repository URL through GitHub will look like `https://github.com/username/name_of_repository.git`.
4. While still in the **URL** tab, under **Local path**, click the **Choose...** button to navigate to the directory where you want the repository to be stored locally on your computer.

Next steps: You can now start making edits within your cloned repository. As you make edits GitHub Desktop will recognize the changes and you will then be able to *push* your changes to the repository.